

O'REILLY®

Learning GitHub Actions

Automation and Integration of CI/CD with GitHub



Brent Laster

Foreword by Julian C. Dunn

Learning GitHub Actions

Maximize your software development productivity with GitHub Actions, the continuous integration and automation platform that works seamlessly with GitHub. With this practical book, author Brent Laster shows software developers, SREs, and DevOps engineers how to gain the maximum benefit from Actions. You'll learn how to leverage this feature in your daily work with GitHub to achieve greater simplification, standardization, and automation.

This book explains the platform, components, use cases, implementation, and integration points of actions, so you can leverage them to provide the functionality and features that today's complex pipelines and software development processes need. You'll learn how to design and implement automated workflows that respond to common events like pushes, pull requests, and review updates—and how to use Actions to implement functionality from simple validation to complex pipelines.

You'll learn how to:

- Understand the structure, syntax, and semantics of GitHub Actions
- Incorporate actions into your automation and processes
- Create custom actions with Docker, JavaScript, or shell approaches
- Troubleshoot and debug workflows that use actions
- Combine actions with GitHub APIs and other integration options
- Securely implement workflows with GitHub Actions
- Migrate from using other CI/CD platforms to GitHub Actions

"Whether you are new to CI/CD and starting with GitHub Actions as your first product in this space or are already a CI/CD expert and migrating from another tool, Brent's book has the right balance of information to help you become productive quickly."

—Julian C. Dunn
Senior Director of Project
Management, GitHub Actions

"If you're looking to master automation in software development, *Learning GitHub Actions* is a comprehensive guide you can't miss."

—Taylor Dolezal
Head of Ecosystem, CNCF

Brent Laster is an R&D DevOps director at SAS. He is a global trainer, author, and speaker on open source technologies.

GITHUB

US \$65.99 CAN \$82.99

ISBN: 978-1-098-13107-4



Twitter: @oreillymedia
linkedin.com/company/oreilly-media
youtube.com/oreillymedia

Learning GitHub Actions

*Automation and Integration
of CI/CD with GitHub*

Brent Laster

Foreword by Julian C. Dunn

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Learning GitHub Actions

by Brent Laster

Copyright © 2023 Tech Skills Transformations, LLC. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<https://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: John Devins

Development Editor: Michele Cronin

Production Editor: Jonathon Owen

Copyeditor: Piper Editorial Consulting, LLC

Proofreader: Kim Wimpsett

Indexer: Ellen Troutman-Zaig

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

August 2023: First Edition

Release History for the First Edition

2023-08-16: First Release

See <https://oreilly.com/catalog/errata.csp?isbn=9781098131074> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Learning GitHub Actions*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-13107-4

[LSI]

To all my family and friends who have helped me write the best chapters of my life.

Table of Contents

Foreword.....	xi
---------------	----

Preface.....	xiii
--------------	------

Part I. Foundations

1. The Basics.....	3
What Is GitHub Actions?	4
Automation Platform	4
Framework	5
What Are the Use Cases for GitHub Actions?	6
Starter Workflows	7
Actions Marketplace	8
What Costs Are Involved?	10
The Free Model	10
The Paid Model	10
When Does Moving to GitHub Actions Make Sense?	13
Investment in GitHub	13
Use of Public Actions	13
Creating Your Own Actions	13
Artifact Management	13
Action Management	14
Conclusion	14
2. How Does Actions Work?.....	15
An Overview	15
Triggering Workflows	17

Components	19
Steps	19
Runners	20
Jobs	20
Workflow	21
Workflow Execution	22
Conclusion	24
3. What's in an action?.....	25
The Structure of an action	26
Interfacing with actions	29
Using actions	32
Public actions and the Marketplace	32
Conclusion	36
4. Working with Workflows.....	37
Creating the First Workflow in a Repository	37
Committing the Initial Workflow	42
Using the VS Code GitHub Actions Extension	59
Conclusion	65
5. Runners.....	67
GitHub-Hosted Runners	67
What's in the Runner Images?	69
Adding Additional Software on Runners	71
Self-Hosted Runners	72
Requirements for Self-Hosted Runners	73
Limits for Self-Hosted Runners	74
Security Considerations for Using Self-Hosted Runners	74
Setting Up a Self-Hosted Runner	75
Using a Self-Hosted Runner	78
Using Labels with Self-Hosted Runners	79
Troubleshooting Self-Hosted Runners	80
Removing a Self-Hosted Runner	82
Autoscaling Self-Hosted Runners	86
Just-in-Time Runners	86
Conclusion	86

Part II. Building Blocks

6. Managing Your Workflow Environments.....	91
Naming Your Workflow and Workflow Runs	91
Contexts	93
Environment Variables	94
Default Environment Variables	95
Secrets and Configuration Variables	97
Managing Permissions for Your Workflow	103
Deployment Environments	105
Conclusion	115
7. Managing Data Within Workflows.....	117
Working with Inputs and Outputs in Workflows	118
Defining and Referencing Workflow Inputs	118
Capturing Output from a Step	119
Capturing Output from a Job	120
Capturing Output from an Action Used in a Step	121
Defining Artifacts	122
Uploading and Downloading Artifacts	123
Adding Parameters	127
Using Caches in GitHub Actions	135
Using the Explicit Cache Action	136
Monitoring Caches	141
Activating a Cache with a Setup Action	143
Conclusion	145
8. Managing Workflow Execution.....	147
Advanced Triggering from Changes	147
Triggering Based on Activity Types	148
Using Filters to Refine Triggers	150
Triggering Workflows Without a Change	152
Dealing with Concurrency	156
Running a Workflow with a Matrix	159
Workflow Functions	160
Conditionals and Status Functions	161
Conclusion	163

Part III. Security and Monitoring

9. Actions and Security.....	167
Security by Configuration	168
Managing Execution of Workflows from Pull Requests	173
Workflow Permissions	174
The CODEOWNERS File	175
Protected Tags	176
Protected Branches	177
Repository Rules	178
Security by Design	181
Secrets	181
Securing Secrets	182
Tokens	183
Dealing with Untrusted Input	191
Securing Your Dependencies	199
Security by Monitoring	201
Scanning	202
Processing Pull Requests Securely	205
Vulnerabilities with Workflows in Pull Requests	207
Vulnerabilities with Source Code in Pull Requests	212
Adding a Pull Request Validation Script	215
Safely Handling Pull Requests	218
Conclusion	220
10. Monitoring, Logging, and Debugging.....	223
Gaining More Observability	223
Understanding Status at a High Level	224
Creating Status Badges for Workflows	226
Working with Past States	232
Mapping Workflow Versions to Runs	232
Re-running Jobs in a Workflow	234
Debugging Workflows	241
Step Debug Logging	242
Debugging the Runner Environment	245
Activating Debugging	248
Augmenting and Customizing Logging	249
Adding Your Own Messages in Logs	249
Additional Log Customizations	251
Creating a Customized Job Summary	253
Conclusion	256

Part IV. Advanced Topics

11. Creating Custom actions.....	259
Anatomy of an action	260
Types of Actions	263
Composite Action	263
Docker Container Action	267
Creating a JavaScript Action	271
Completing Your Action Creation	274
Publishing Actions on the GitHub Marketplace	276
Updating Actions on the Marketplace	282
Removing an Action from the Marketplace	285
The Actions Toolkit	285
Using Workflow Commands from the Toolkit	286
Local actions	289
Conclusion	291
12. Advanced Workflows.....	293
Creating Your Own Starter Workflows	293
Creating a Starter Workflow Area	295
Creating a Starter Workflow File	295
Adding Supporting Pieces	297
Using the New Starter Workflow	299
Reusable Workflows	300
Inputs and Secrets	303
Outputs	305
Limitations	306
Required Workflows	308
Constraints	308
Example	309
Execution	313
Conclusion	315
13. Advanced Workflow Techniques.....	317
Driving GitHub from Your Workflow	317
Using the GitHub CLI	318
Creating Scripts	319
Invoking GitHub APIs	320
Using a Matrix Strategy to Automatically Create Jobs	322
One-Dimensional Matrices	322
Multi-dimensional Matrices	323
Including Extra Values	327

Excluding Values	328
Handling Failure Cases	329
Defining Max Concurrent Jobs	330
Using Containers in Your Workflow	330
Using a Container as the Environment for a Job	331
Using a Container with a Step	333
Running Containers as Services in a Job	334
Conclusion	335
14. Migrating to GitHub Actions.	337
Prep	338
Source Code	338
Automation	340
Infrastructure	340
Users	341
Azure Pipelines	342
CircleCI	344
GitLab CI/CD	346
Jenkins	348
Travis CI	351
GitHub Actions Importer	353
Authentication	354
Planning	355
Build Steps and Related	357
Manual Tasks	359
File Manifest	360
Forecasting	361
Doing a Dry Run	363
Creating Custom Transformers for the Importer	364
Doing the Actual Migration	369
Conclusion	373
Index.	375

Foreword

The fundamental concepts of continuous integration/continuous delivery (CI/CD) have now been around for several decades, since Martin Fowler and Matthew Foemmel of Thoughtworks first popularized CI in **their seminal essay** of September 2000, and Jez Humble and Dave Farley wrote about CD in their 2010 book *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation* (Addison-Wesley Professional). Yet it has taken years for widespread adoption of tools for CI/CD and for the notion of a software delivery pipeline to take root. I believe this is because three fundamental socio-technical changes in how we do software development had to occur first:

- Development had to become collaborative, rather than performed by isolated, individual engineers. This was driven first by a truly distributed version control system (Git) and then accelerated through pull-request platforms like GitHub.
- Widespread adoption of agile practices needed to occur. Motivated by metrics from DevOps leaders like DevOps Research Associates (DORA) that, contrary to intuition, showed more frequent delivery of smaller changes reduces risk, savvy engineering leaders drove the implementation of frequently used build pipelines where software was continuously built, tested, and deployed directly to customers—sometimes dozens of times per day.
- The overburdening of traditional IT operations functions with increased complexity drove massive cultural changes via the DevOps movement. This led to a you-build-it, you-own-it approach to software operations where, increasingly, developers take full ownership for the success, failure, and performance of their software in production, rather than throwing code over the wall to a release engineering team that would operate a build process and somehow “add quality” to software that didn’t have it already.

All these changes mean that GitHub Actions, as a relatively recent entrant to the CI/CD pipeline and automation category, is a substantially different product from

incumbents. It is natively integrated into GitHub, making it a natural fit for developers who are already familiar with storing their source code there. GitHub Actions is also designed around the concept of a workflow, which can be used to create CI/CD pipelines but also handle any other kind of software automation tasks like managing open issues and tasks that open source and enterprise developers alike need to perform in the course of their work. Finally, GitHub Actions, as the name would suggest, is based on the notion of an action: a reusable component that helps to encapsulate common tasks and reduce repetition when authoring workflows. The GitHub Marketplace offers nearly 20,000 actions at the time of writing, making it easy for developers, DevOps engineers, and site reliability engineers to get started with any kind of build automation task.

Although GitHub Actions is a sophisticated product, learning it doesn't need to be complicated. Brent Laster has written an excellent book that relies on progressive disclosure, starting with the most basic concepts to get you up and running with GitHub Actions while also providing a comprehensive tour of GitHub Actions' most advanced features to help you to optimize your use of the product as you adopt it across your organization. My team and I have been delighted to partner with Brent in ensuring that the content covered here is as current as possible. Whether you are new to CI/CD and starting with GitHub Actions as your first product in this space or are already a CI/CD expert and migrating from another tool, Brent's book has the right balance of information to help you become productive quickly.

We wish you the greatest success in automating your software delivery processes.

— *Julian C. Dunn*
Senior Director of Product Management
GitHub Actions

Preface

Releasing software should be easy.... Automate almost everything, and keep everything you need to build, deploy, test, and release your application in version control.

—David Farley, *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation*

Back in 1968, the London Underground in the United Kingdom needed a digital sign to warn passengers to be careful while crossing the gaps between train doors and station platforms. Since data storage for such signs was very expensive back in the day, they chose a very short phrase to help keep riders alert: “mind the gap.”

These days, the word “mind” is less commonly used, but the intent to bring awareness to missing parts or things that can trip you up and to act on them is still meaningful. And it is just as important when we apply the idea to business and technical processes that can benefit from automation.

From its inception in 2008, GitHub has filled gaps in terms of allowing users to collaborate and build communities around open source software. And it has done this very well. It is challenging not to overestimate the significance of the SaaS hosting model that GitHub pioneered and the collaborative ecosystem it has built around it. Yet up until a few years ago, there was one key piece of that ecosystem that was clearly missing—a tightly integrated automation platform for key functions like CI/CD.

Certainly, there has been no shortage of applications that have worked to fill that gap. Tools such as Jenkins, Travis CI, CircleCI, Azure DevOps, and more have provided integration methods through various approaches such as webhooks. However, users of GitHub still had to go *outside* their collaboration environment to use another application to get the basic functionality they needed. All of that has changed with the addition of GitHub Actions.

Actions is challenging to classify with a standalone designation. It is a logical extension of the larger GitHub model. And while this is not a general book on GitHub, I have tried to write it in such a way that you can see how GitHub Actions plays with